📋 Part 4: Rate Limiting — Notes

WHAT IS RATE LIMITING?

Rate limiting controls how many API requests a user can make in a given time period. Think of it like a restaurant:

"Each customer can order maximum 20 dishes per minute.

If they try to order a 21st dish, we say: Please wait."

For your API:

"Each user (identified by IP address) can make 20 API calls per minute.

On the 21st call, the API returns: 429 Too Many Requests."

WHY YOUR RAG PROJECT NEEDS IT

Without Rate Limiting:

├─ Someone writes a bot/script

├─ Sends 1000 queries in 1 minute

├─ Each query calls Gemini API (costs money!)

├─ Your Gemini bill: 💸 💸 💸 💸 💸

├─ Server overloaded, CPU maxed out

└─ Other users can't use the app

With Rate Limiting:

├─ Bot tries 1000 queries in 1 minute

├─ First 20 go through ✅

├─ 21st request → "429 Too Many Requests, wait 60 seconds"

├─ Gemini bill stays normal

├─ Server stays healthy

└─ Other users are unaffected ✅

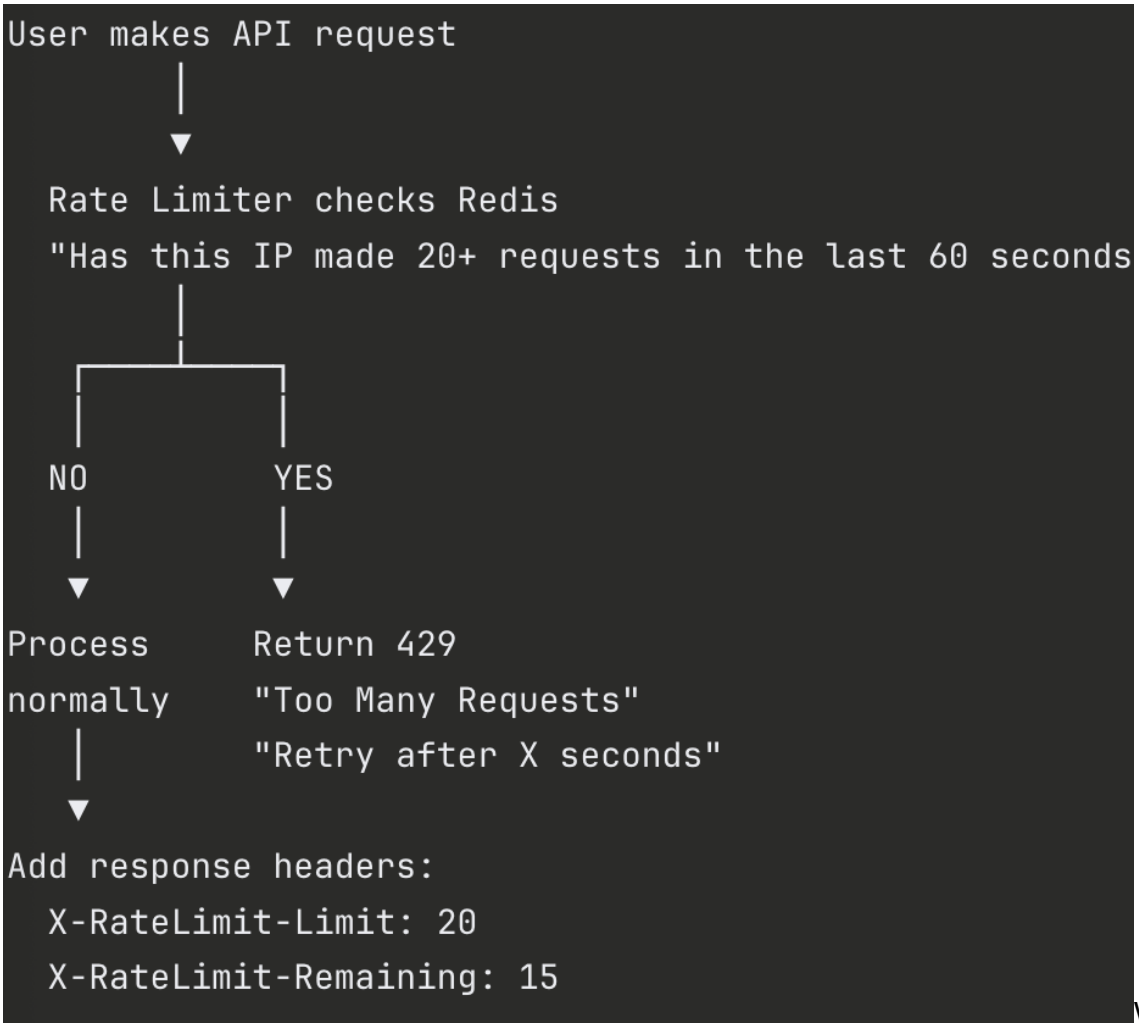
Three things you're protecting:

1. Gemini API costs → Every /api/query = 1 Gemini call = money

2. Server resources → Each query uses CPU for embeddings

3. Database → Too many queries = PostgreSQL overwhelmed

HOW IT WORKS



Why

Redis? Because it's:

- Super fast (checking takes <1ms)

- Has built-in expiration (counter auto-resets after 60 seconds)
- Already running in your Docker setup

FILES CREATED / MODIFIED

```
NEW FILES:
└── app/rate_limiter.py     → Rate limiting middleware

MODIFIED FILES:
├── app/main.py             → Added RateLimitMiddleware
└── app/api/routes.py       → Added /api/rate-limit-status endpoint
```

HOW THE CODE WORKS

app/rate_limiter.py

This is a middleware — it runs BEFORE every API request reaches your endpoint:

```
Request comes in
    |
    ▼
Rate Limit Middleware ← checks Redis
    |
    ▼
Request Logging Middleware ← logs the request
    |
    ▼
Your actual endpoint ← processes the query
```

Key logic:

```
python# 1. Get user's IP address
```

```
client_ip = request.client.host


# 2. Create Redis key like "ratelimit:192.168.1.1"
rate_key = f"ratelimit:{client_ip}"


# 3. Check how many requests this IP has made
current_count = redis.get(rate_key)


# 4. If first request → start counting, expire after 60 seconds
if current_count is None:
   redis.setex(rate_key, 60, 1)    # key, expiry, count


# 5. If over limit → reject with 429
elif current_count >= 20:
   return 429 "Too Many Requests"


# 6. If under limit → increment and allow
else:
   redis.incr(rate_key)          # count goes up by 1
```

Smart Design Decisions

1. Skips health checks and docs

   → /health, /docs, /openapi.json are NOT rate limited

   → Monitoring tools need unrestricted access

2. Graceful degradation

   → If Redis is down, requests go through WITHOUT rate limiting

→ Better to serve without limits than to block everyone

3. IP-based tracking

→ Each IP gets its own counter

→ Supports X-Forwarded-For header for users behind proxies

4. Response headers

→ X-RateLimit-Limit: 20 (your limit)

→ X-RateLimit-Remaining: 15 (how many left)

→ Client apps can read these to show warnings

API ENDPOINTS

GET /api/rate-limit-status

→ Shows current rate limiting info

→ Response:


```
{
  "rate_limiting": "enabled",
  "limit": "20 requests per minute",
  "active_users": 1,
  "details": [
    {
      "ip": "167.82.60.223",
      "requests_made": 15,
      "resets_in_seconds": 45
    }
  ]
```

```
}
```

HOW TO TEST

Step 1: Open http://localhost:8000/api/documents

Step 2: Refresh rapidly 20+ times

Step 3: On 21st request you'll see:

```
{"error": "Too many requests", "retry_after_seconds": 45}
```

Step 4: Wait 60 seconds → counter resets → works again

Step 5: Check logs → WARNING | rate_limiter | Rate limited: 172.18.0.1

INTERVIEW TALKING POINTS

"I implemented Redis-based API rate limiting as middleware to protect the system from abuse. Each client IP is limited to 20 requests per minute using Redis counters with automatic expiration. The middleware adds standard rate limit headers (X-RateLimit-Limit, X-RateLimit-Remaining) to every response so client applications can track their usage. It gracefully degrades — if Redis is unavailable, requests pass through rather than blocking all users. Health check and documentation endpoints are excluded from rate limiting so monitoring tools always have access."

Key terms:

- Redis-based rate limiting
- Sliding window / fixed window counter
- Middleware pattern
- Graceful degradation
- Rate limit headers
- IP-based tracking
- 429 Too Many Requests

QUICK REFERENCE

Config:

Limit: 20 requests per minute per IP

Window: 60 seconds (auto-resets)

Storage: Redis (ratelimit:{ip} keys)

Skipped paths:

/health, /docs, /openapi.json, /redoc

Response when limited:

Status: 429

Body: {"error": "Too many requests", "retry_after_seconds": N}

Response headers (every request):

X-RateLimit-Limit: 20

X-RateLimit-Remaining: N

Step 1: Create Rate Limiter

Path: backend/app/rate_limiter.py (new file inside app/ folder)

python

```
"""
Rate Limiting using Redis.
Limits how many API requests a user can make per minute.
```

```
Prevents abuse, protects Gemini API costs, and keeps server healthy.
"""
from starlette.middleware.base import BaseHTTPMiddleware

from starlette.requests import Request
from starlette.responses import JSONResponse
from core.cache import get_cache_manager
from core.logger import get_logger
import time

logger = get_logger("rate_limiter")


class RateLimitMiddleware(BaseHTTPMiddleware):
    def __init__(self, app, requests_per_minute: int = 20):
        super().__init__(app)
        self.requests_per_minute = requests_per_minute
        self.window_seconds = 60
        logger.info(f"Rate limiter initialized: {requests_per_minute} requests/minute")

    def _get_client_ip(self, request: Request) -> str:
        """Get the client's IP address."""
        # Check for forwarded header (when behind a proxy/load balancer)
        forwarded = request.headers.get("X-Forwarded-For")
        if forwarded:
            return forwarded.split(",")[0].strip()
        return request.client.host

    async def dispatch(self, request: Request, call_next):
        # Skip rate limiting for health checks and docs
        skip_paths = ["/health", "/docs", "/openapi.json", "/redoc"]
        if any(request.url.path.startswith(path) for path in skip_paths):
            return await call_next(request)

        # Get Redis client
        cache = get_cache_manager()

        # If Redis is not available, let the request through
```

```
        # (don't block users just because Redis is down)
        if not cache.enabled or not cache.redis_client:
            return await call_next(request)

        try:
            # Create a unique key for this user based on IP
            client_ip = self._get_client_ip(request)
            rate_key = f"ratelimit:{client_ip}"

            # Get current request count
            current_count = cache.redis_client.get(rate_key)

            if current_count is None:
                # First request — start counting
                cache.redis_client.setex(rate_key, self.window_seconds, 1)
                remaining = self.requests_per_minute - 1
            else:
                current_count = int(current_count)

                if current_count >= self.requests_per_minute:
                    # RATE LIMITED — too many requests
                    ttl = cache.redis_client.ttl(rate_key)
                    logger.warning(
                        f"Rate limited: {client_ip} | "
                        f"{current_count}/{self.requests_per_minute} requests | "
                        f"retry in {ttl}s"
                    )
                    return JSONResponse(
                        status_code=429,
                        content={
                            "error": "Too many requests",
                            "detail": f"Rate limit: {self.requests_per_minute} requests per minute",
                            "retry_after_seconds": ttl
                        }
                    )

                # Increment counter
```

```
            cache.redis_client.incr(rate_key)
            remaining = self.requests_per_minute - current_count - 1

        # Process the request
        response = await call_next(request)

        # Add rate limit headers to response
        response.headers["X-RateLimit-Limit"] = str(self.requests_per_minute)
        response.headers["X-RateLimit-Remaining"] = str(max(0, remaining))

        return response

    except Exception as e:
        # If rate limiting fails, don't block the request
        logger.error(f"Rate limiter error: {e}")
        return await call_next(request)
```

Step 2: Update main.py to Add Rate Limiter

Path: backend/app/main.py — replace entire file with:

python

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.config import get_settings
from app.models import HealthCheck
from app.api import routes
from app.api import health
from app.middleware import RequestLoggingMiddleware
from app.rate_limiter import RateLimitMiddleware
from core.logger import get_logger
import os

settings = get_settings()
logger = get_logger("main")
```

```
app = FastAPI(
    title="Time-Aware RAG API",
    description="Retrieval Augmented Generation with Temporal Intelligence",
    version="2.0.0"
)

# CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Rate limiting middleware (outermost — checks FIRST before anything else)
app.add_middleware(RateLimitMiddleware, requests_per_minute=20)

# Request logging middleware
app.add_middleware(RequestLoggingMiddleware)

# Include routes
app.include_router(routes.router)
app.include_router(health.router)

@app.on_event("startup")
async def startup_event():
    logger.info("🚀 Time-Aware RAG API starting up...")
    logger.info(f"📦 Embedding model: {settings.embedding_model}")
    logger.info(f"🗄️ Database: connected")
    logger.info(f"💾 Redis caching: {'enabled' if settings.enable_caching else 'disabled'}")
    logger.info(f"🛡️ Rate limiting: 20 requests/minute")
    logger.info("✅ API ready to serve requests")

@app.on_event("shutdown")
async def shutdown_event():
```

```
    logger.info("🛑 Time-Aware RAG API shutting down...")

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(
        "app.main:app",
        host=settings.host,
        port=settings.port,
        reload=True
    )
```

Step 3: Add Rate Limit Stats Endpoint

Path: backend/app/api/routes.py — add this endpoint at the bottom of the file, before the last line (get_stats):

python

```
# ==================== RATE LIMIT INFO ====================

@router.get("/rate-limit-status")
async def get_rate_limit_status():
    """
    🛡️ Check your current rate limit status.
    Shows how many requests you have remaining.
    """
    try:
        cache = get_cache_manager()

        if not cache.enabled or not cache.redis_client:
            return {
                "rate_limiting": "disabled",
                "reason": "Redis not available"
            }

        # Count all rate limit keys
```

```
    rate_keys = cache.redis_client.keys("ratelimit:*")

    active_users = []
    for key in rate_keys:
        count = cache.redis_client.get(key)
        ttl = cache.redis_client.ttl(key)
        ip = key.split(":", 1)[1] if ":" in key else key
        active_users.append({
            "ip": ip,
            "requests_made": int(count) if count else 0,
            "resets_in_seconds": ttl
        })

    return {
        "rate_limiting": "enabled",
        "limit": "20 requests per minute",
        "active_users": len(active_users),
        "details": active_users
    }
  except Exception as e:
    return {"error": str(e)}
```

That's it — just 2 new files and 1 small update. Now rebuild:

bash

```
docker-compose down
docker-compose up --build
```

ERROR HANDLING

If you guys face any issues due cache

```
docker-compose down
```

bash

```
docker builder prune -f
```

bash

```
docker-compose up --build
```

This clears the corrupted cache and forces a fresh build. It'll take 2-3 minutes again since it's rebuilding from scratch. Share what happens!